MINISTRY OF EDUCATION, CULTURE AND RESEARCH OF THE REPUBLIC OF MOLDOVA
Technical University of Moldova
Faculty of Computers, Informatics and Microelectronics
Department of Software Engineering and Automation

Ion Moroșanu FAF-242

Report

Laboratory Work no. 3
Cryptography and Security

Checked by:
Aureliu Zgureanu
FCIM, UTM

Chișinău – 2026

TABLE OF CONTENTS

INTRODUCTION **3**
Purpose of the work 3
The task 3
Theoretical notes 3
1 ADAPTING THE CIPHER TO ROMANIAN **5**
1.1 The alphabet and the size of the matrix 5
1.2 Filler letters 5
1.3 Letters with diacritics 5
1.4 Validation of the input 5
2 IMPLEMENTATION **7**
3 TESTING **11**
3.1 Check on the example of the guide 11
3.2 Encryption of a Romanian message 11
3.3 Decryption 13
3.4 Other checks of the input 14
CONCLUSIONS **15**
REFERENCES **16**

INTRODUCTION

Purpose of the work

The purpose of the third laboratory work is to study polyalphabetic substitution ciphers, the Vigenère cipher and the Playfair cipher, and to implement one of them for messages written in Romanian [1].

The task

The guide gives two tasks: students with an odd number in the group register implement the Playfair cipher (task 3.1), and students with an even number implement the Vigenère cipher (task 3.2). My number in the register is 15, so this report covers task 3.1. The program has to meet the following requirements [1]:

a) it encrypts and decrypts messages in Romanian, whose alphabet has 31 letters;

b) the text may contain only letters, 'A'–'Z' and 'a'–'z'; if the user types any other character, the program shows the correct range;

c) the key has at least 7 letters;

d) the user chooses the operation, encryption or decryption, types the key and the message or the cryptogram, and gets the cryptogram or the decrypted message;

e) the final step of decryption, removing the filler letters and putting the spaces back, is done by hand.

Theoretical notes

A monoalphabetic cipher replaces a letter with the same letter every time, so the frequencies of the plaintext pass straight into the cryptogram; this is how the cipher of Lab 2 was broken. Polyalphabetic ciphers change the substitution of a letter along the text, depending on its position or on its context, and in this way they flatten the frequency distribution [1].

The Playfair cipher was invented by Charles Wheatstone and described in 1854. It carries the name of his friend Baron Lyon Playfair, who promoted it to the British government. It was used by the British army in the Boer War and, in modified forms, in both world wars [1], [2]. Playfair encrypts pairs of letters (digraphs) instead of single letters. The key is written into a 5×5 matrix without repeated letters, and the rest of the alphabet fills the remaining cells; J is written as I so that 25 letters fit.

The message is prepared first: it is written in capital letters without spaces or punctuation and split into pairs, a filler letter (Q, X or Z) is put between two equal letters that would fall into the same

pair, and a filler letter is added at the end if the number of letters is odd. Each pair is then replaced according to three rules [1]:

1) if the two letters are in different rows and columns, each one is replaced by the letter in its own row and in the column of the other letter, that is, by the opposite corner of the rectangle they form;

2) if they are in the same row, each one is replaced by the letter to its right, and the last letter of the row by the first one;

3) if they are in the same column, each one is replaced by the letter below it, and the last letter of the column by the first one.

Decryption uses the same matrix. The rectangle rule stays the same, and in rules 2 and 3 the letters move to the left and up. Because the substitution of a letter depends on the other letter of its pair, the same plaintext letter is encrypted into different letters in different places. The cipher is still a substitution of pairs, though, so a frequency analysis of pairs of letters can break it, but with 600 possible pairs instead of 26 letters it needs much more text.

1 ADAPTING THE CIPHER TO ROMANIAN

1.1 The alphabet and the size of the matrix

The Romanian alphabet has 31 letters: `A Ă Â B C D E F G H I Î J K L M N O P Q R S Ș T Ț U V W X Y Z`. A 5×5 matrix holds only 25 of them, so a larger matrix is needed, which the guide allows [1]. Two shapes were considered:

a) a 6×6 matrix with 36 cells: all 31 letters fit, but 5 cells remain and would have to be filled with other symbols. Such a symbol could then appear in a cryptogram, which would contain characters outside the allowed range;

b) a 5×6 matrix with 30 cells: one letter has to be left out, and the guide gives the way to do it, writing J as I, as in the original Playfair [1].

The program uses the 5×6 matrix (5 rows and 6 columns) and writes J as I. J is one of the rare letters of Romanian, and after decryption it is easy to put back from the context; for example, `IOI` is read as joi. The 30 remaining letters fill the matrix exactly, with no free cells, so a cryptogram contains only letters of the alphabet. Because the matrix is not square, the shifts are taken modulo 6 along a row and modulo 5 along a column.

1.2 Filler letters

The filler letter is X, the letter that the guide uses in its own example, where FR EE DO M becomes FR EX ED OM [1]. X is rare in Romanian, so a filler is easy to notice in the decrypted text. When the letter that needs a filler is X itself, the filler is Q, because a pair XX would break the rule again. The same fillers complete a message with an odd number of letters.

1.3 Letters with diacritics

Ș and Ț exist in two forms in Unicode: with a comma below (U+0218 and U+021A), which is the correct Romanian form, and with a cedilla (U+015E and U+0162), which the legacy Romanian keyboard layout of Windows still types. The program accepts both and turns the cedilla forms into the comma forms, so each letter has exactly one place in the matrix. The text is also normalized to the NFC form [3], so that a letter typed as a base letter followed by a combining diacritic is treated as the single letter it represents.

1.4 Validation of the input

The guide asks for Romanian messages of 31 letters and at the same time names the ranges 'A'–'Z' and 'a'–'z', so the two requirements were read together: the letters of the Romanian alphabet are accepted in either case, and every other character (digits, punctuation, symbols) is rejected. Spaces

are accepted and removed, as in the preparation step of the guide. If the input contains another character, the program lists the wrong characters, shows the allowed range and asks again. It checks three more conditions:

a) the key must have at least 7 letters, spaces not counted;

b) a cryptogram must have an even number of letters, because it is made of pairs;

c) a cryptogram cannot contain a pair of two equal letters, since none of the three rules can produce one: the two letters of a pair always end up in different cells.

2 IMPLEMENTATION

The program is written in Python 3.12 in one file, `playfair.py`, and uses only the standard modules `sys` and `unicodedata`. It runs in the console and is started with `python playfair.py`. The constants at the beginning of the file hold the alphabet without J (`ALPHABET`), the size of the matrix (`ROWS = 5`, `COLS = 6`), the filler letters and the minimum length of the key. Table 2.1 lists the functions.

Table 2.1 – The functions of the program

Function	Purpose
`invalid_characters`	returns the characters of the text that are not allowed
`letters_only`	normalizes the text (NFC form, comma diacritics), converts it to capitals, writes J as I and removes the spaces
`build_matrix`	writes the letters of the key without repetitions, then the rest of the alphabet, into the 5×6 matrix
`find_positions`	returns the row and the column of every letter of the matrix
`make_pairs`	splits the message into pairs and inserts the filler letters
`split_cryptogram`	splits a cryptogram into pairs and checks it
`transform_pair`	applies the three rules to one pair, with a shift of +1 or −1
`encrypt`, `decrypt`	transform all the pairs with the shift +1 or −1
`read_letters`, `read_choice`	read the input and repeat the question until it is valid
`run_encryption`, `run_decryption`	read the message or the cryptogram and print the result
`main`	the menu: the operation, the key, the matrix and the result, repeated until the user exits

The key matrix is built by going through the letters of the key followed by the whole alphabet and keeping each letter the first time it appears; the 30 distinct letters are then cut into 5 rows of 6. For example, the key Securitatea informației gives SECURITANFOMȚ, followed by the letters of the alphabet that are not in it.

Encryption and decryption share one function, `transform_pair`. Its parameter `step` is +1 for encryption and −1 for decryption: in the same row the column changes by `step` modulo 6, in the same column the row changes by `step` modulo 5, and in the rectangle case the two letters swap columns, which is the same in both directions. The `%` operator of Python always returns a non-negative result, so (0 − 1) % 6 = 5, and when decrypting, the first letter of a row is replaced by the last one.

Listing 2.1 shows the full source code of the program.

Listing 2.1 – playfair.py

```
import sys
import unicodedata

ALPHABET = "AĂÂBCDEFGHIÎKLMNOPQRSȘTȚUVWXYZ"
ROWS = 5
COLS = 6
FILLER = "X"
SPARE_FILLER = "Q"
MIN_KEY_LENGTH = 7
ALLOWED_RANGE = "'A'-'Z', 'a'-'z' and Ă Â Î Ș Ț (ă â î ș ț)"
VALID_CHARACTERS = set(ALPHABET + ALPHABET.lower() + "Jj" + "ȘșȚț" + " ")
CEDILLA_TO_COMMA = str.maketrans("ȘșȚț", "ȘșȚț")
MENU = "Operation (1 - encrypt, 2 - decrypt, 0 - exit): "


def invalid_characters(text):
    text = unicodedata.normalize("NFC", text)
    return sorted({ch for ch in text if ch not in VALID_CHARACTERS})


def letters_only(text):
    text = unicodedata.normalize("NFC", text).translate(CEDILLA_TO_COMMA)
    return text.upper().replace("J", "I").replace(" ", "")


def build_matrix(key):
    order = []
    for ch in letters_only(key) + ALPHABET:
        if ch not in order:
            order.append(ch)
    return [order[r * COLS:(r + 1) * COLS] for r in range(ROWS)]


def find_positions(matrix):
    positions = {}
    for r, row in enumerate(matrix):
        for c, ch in enumerate(row):
            positions[ch] = (r, c)
    return positions


def make_pairs(text):
    pairs = []
    i = 0
    while i < len(text):
        first = text[i]
        second = text[i + 1] if i + 1 < len(text) else None
        if second is None or second == first:
            pairs.append(first + (SPARE_FILLER if first == FILLER else FILLER))
            i += 1
        else:
            pairs.append(first + second)
            i += 2
    return pairs


def split_cryptogram(text):
    if len(text) % 2 == 1:
        return None, f"The number of letters must be even, it is {len(text)}."
    pairs = [text[i:i + 2] for i in range(0, len(text), 2)]
    doubles = [p for p in pairs if p[0] == p[1]]
    if doubles:
        error = "A cryptogram cannot have a pair of equal letters: "
        return None, error + " ".join(doubles)
    return pairs, None
```

```
def transform_pair(pair, matrix, positions, step):
    (r1, c1), (r2, c2) = positions[pair[0]], positions[pair[1]]
    if r1 == r2:
        return matrix[r1][(c1 + step) % COLS] + matrix[r2][(c2 + step) % COLS]
    if c1 == c2:
        return matrix[(r1 + step) % ROWS][c1] + matrix[(r2 + step) % ROWS][c2]
    return matrix[r1][c2] + matrix[r2][c1]


def encrypt(pairs, matrix):
    positions = find_positions(matrix)
    return [transform_pair(p, matrix, positions, 1) for p in pairs]


def decrypt(pairs, matrix):
    positions = find_positions(matrix)
    return [transform_pair(p, matrix, positions, -1) for p in pairs]


def read_letters(prompt, what, min_length=1):
    while True:
        text = input(prompt)
        bad = invalid_characters(text)
        if bad:
            print("Invalid characters: " + ", ".join(f"'{ch}'" for ch in bad))
            print(f"Use only {ALLOWED_RANGE}. Spaces are ignored.")
            continue
        letters = letters_only(text)
        if len(letters) < min_length:
            print(f"The {what} must have at least {min_length} letters, "
                  f"it has {len(letters)}.")
            continue
        return letters


def read_choice():
    while True:
        choice = input(MENU).strip()
        if choice in ("0", "1", "2"):
            return choice
        print("Type 1, 2 or 0.")


def print_matrix(matrix):
    print("Matrix:")
    for row in matrix:
        print("    " + " ".join(row))


def run_encryption(matrix):
    text = read_letters("Message: ", "message")
    pairs = make_pairs(text)
    print("Prepared message: " + " ".join(pairs))
    print("Cryptogram: " + "".join(encrypt(pairs, matrix)))


def run_decryption(matrix):
    while True:
        text = read_letters("Cryptogram: ", "cryptogram", 2)
        pairs, error = split_cryptogram(text)
        if error is None:
            break
        print(error)
    result = decrypt(pairs, matrix)
    print("Decrypted pairs: " + " ".join(result))
    print("Decrypted message: " + "".join(result))
    print(f"Remove the filler letters {FILLER}/{SPARE_FILLER} "
```

```
          "and add the spaces by hand.")


def main():
    sys.stdin.reconfigure(encoding="utf-8")
    sys.stdout.reconfigure(encoding="utf-8")
    print("Playfair cipher for Romanian (31 letters, J is written as I)")
    while True:
        choice = read_choice()
        if choice == "0":
            break
        key = read_letters("Key: ", "key", MIN_KEY_LENGTH)
        matrix = build_matrix(key)
        print_matrix(matrix)
        if choice == "1":
            run_encryption(matrix)
        else:
            run_decryption(matrix)
        print()


if __name__ == "__main__":
    try:
        main()
    except (EOFError, KeyboardInterrupt):
        print()
```

3 TESTING

3.1 Check on the example of the guide

Before using the program on Romanian texts, its functions were checked on the English example of the guide, where the cryptogram is known [1]. A short script imported the program, replaced the alphabet with the 25 English letters without J and the size of the matrix with 5×5, built the matrix of the key First Amendment, encrypted the 115 prepared pairs of the guide and decrypted the result (Listing 3.1, where the long text and cryptogram are shortened).

Listing 3.1 – The check on the example of the guide

```
import playfair as pf

guide_pairs = """CO NG RE SX SZ SH AL LM MA KE NO ... RI EV AN CE SB""".split()
guide_cipher = ("OWEHEGRYTYNQBVOAEMGDMQVBXINRXGKISMBEDNTFBLOF" ... "MFH")

pf.ALPHABET = "ABCDEFGHIKLMNOPQRSTUVWXYZ"
pf.ROWS, pf.COLS = 5, 5
matrix = pf.build_matrix("First Amendment")
pf.print_matrix(matrix)
cipher = "".join(pf.encrypt(guide_pairs, matrix))
print("Same cryptogram as the guide:", cipher == guide_cipher, len(cipher))
pairs = pf.split_cryptogram(cipher)[0]
print("Decrypted to the prepared text:", pf.decrypt(pairs, matrix) == guide_pairs)
print(", ".join(p + " -> " + pf.encrypt([p], matrix)[0] for p in ("NP", "IT", "CW")))
```

Listing 3.2 – The output of the check

```
Matrix:
    F I R S T
    A M E N D
    B C G H K
    L O P Q U
    V W X Y Z
Same cryptogram as the guide: True 230
Decrypted to the prepared text: True
NP -> EQ, IT -> RF, CW -> OI
```

The matrix is the same as in the guide, the 230 letters of the cryptogram are identical to those of the guide, decryption gives back the prepared text, and the three pairs that the guide gives as examples of the rules (NP, IT and CW) are encrypted as in the guide. A second test encrypted and decrypted 5000 random Romanian texts with random keys of 7 to 20 letters. Every time, decryption returned exactly the prepared pairs, and no prepared or encrypted pair contained two equal letters.

3.2 Encryption of a Romanian message

The key is Securitatea informației and the message is Copiii se întâlnesc joi la ora șapte în fața școlii. The message was chosen to contain the special cases: the letters Î, Â, Ș and Ț, a J, and three equal letters in a row (copiii).

Without the repeated letters the key becomes SECURITANFOMȚ. Table 3.1 shows the matrix, with the numbers of the rows and of the columns.

Table 3.1 – The matrix of the key Securitatea informației

r / c	0	1	2	3	4	5
0	S	E	C	U	R	I
1	T	A	N	F	O	M
2	Ț	Ă	Â	B	D	G
3	H	Î	K	L	P	Q
4	Ș	V	W	X	Y	Z

After the spaces are removed and J is written as I, the message has 42 letters:

```
COPIIISEÎNTÂLNESCIOILAORAȘAPTEÎNFAȚAȘCOLII
```

In copiii the second and the third I would fall into the same pair, so X is inserted after the second one (IX), and the third I goes with the S of se (IS). This filler makes the number of letters odd (43), so the last I of școlii is left alone and receives the filler X as well. The prepared message has 22 pairs:

```
CO PI IX IS EÎ NT ÂL NE SC IO IL AO RA ȘA PT EÎ NF AȚ AȘ CO LI IX
```

Table 3.2 shows how the first seven pairs are encrypted. Together they use all three rules, including the wrap-around at the end of a row.

Table 3.2 – The encryption of the first seven pairs

Pair	Positions (r, c)	Rule	Result
CO	(0, 2), (1, 4)	rectangle	RN
PI	(3, 4), (0, 5)	rectangle	QR
IX	(0, 5), (4, 3)	rectangle	UZ
IS	(0, 5), (0, 0)	same row; I is the last letter of the row, so it becomes the first one, S	SE
EÎ	(0, 1), (3, 1)	same column	AV
NT	(1, 2), (1, 0)	same row	FA
ÂL	(2, 2), (3, 3)	rectangle	BK

The pairs PI, IX and IS show the polyalphabetic effect: the same letter I is encrypted as R, U and S, depending on the letter it is paired with. On the other hand, the pair EÎ appears twice in the message and is encrypted as AV both times, because Playfair is still a fixed substitution of pairs.

Listing 3.3 shows the whole run in the console. Before the correct input, the user types a key of 5 letters, a key with digits and a message with punctuation and a digit; each time the program explains the problem and asks again.

Listing 3.3 – Encryption in the console

```
Playfair cipher for Romanian (31 letters, J is written as I)
Operation (1 - encrypt, 2 - decrypt, 0 - exit): 1
Key: cheie
The key must have at least 7 letters, it has 5.
Key: Securitate 2026
Invalid characters: '0', '2', '6'
Use only 'A'-'Z', 'a'-'z' and Ă Â Î Ș Ț (ă â î ș ț). Spaces are ignored.
Key: Securitatea informației
Matrix:
    S E C U R I
    T A N F O M
    Ț Ă Â B D G
    H Î K L P Q
    Ș V W X Y Z
Message: Copiii se întâlnesc joi, la ora 7!
Invalid characters: '!', ',', '7'
Use only 'A'-'Z', 'a'-'z' and Ă Â Î Ș Ț (ă â î ș ț). Spaces are ignored.
Message: Copiii se întâlnesc joi la ora șapte în fața școlii
Prepared message: CO PI IX IS EÎ NT ÂL NE SC IO IL AO RA ȘA PT EÎ NF AȚ AȘ CO LI IX
Cryptogram: RNQRUZSEAVFABKACEURMUQNMEOVTHOAVFOTĂTVRNQUUZ

Operation (1 - encrypt, 2 - decrypt, 0 - exit): 0
```

The cryptogram has 44 letters:

```
RNQRUZSEAVFABKACEURMUQNMEOVTHOAVFOTĂTVRNQUUZ
```

3.3 Decryption

Listing 3.4 shows the decryption of the same cryptogram with the same key. The first attempt leaves out the last letter, and the program rejects it, because 43 letters cannot be split into pairs.

Listing 3.4 – Decryption in the console

```
Playfair cipher for Romanian (31 letters, J is written as I)
Operation (1 - encrypt, 2 - decrypt, 0 - exit): 2
Key: Securitatea informației
Matrix:
    S E C U R I
    T A N F O M
    Ț Ă Â B D G
    H Î K L P Q
    Ș V W X Y Z
Cryptogram: RNQRUZSEAVFABKACEURMUQNMEOVTHOAVFOTĂTVRNQUU
The number of letters must be even, it is 43.
Cryptogram: RNQRUZSEAVFABKACEURMUQNMEOVTHOAVFOTĂTVRNQUUZ
Decrypted pairs: CO PI IX IS EÎ NT ÂL NE SC IO IL AO RA ȘA PT EÎ NF AȚ AȘ CO LI IX
Decrypted message: COPIIXISEÎNTÂLNESCIOILAORAȘAPTEÎNFAȚAȘCOLIIX
Remove the filler letters X/Q and add the spaces by hand.

Operation (1 - encrypt, 2 - decrypt, 0 - exit): 0
```

The decrypted text is:

```
COPIIXISEÎNTÂLNESCIOILAORAȘAPTEÎNFAȚAȘCOLIIX
```

The last step is done by hand, as the task requires: the two filler letters X are removed (one inside COPIII and one at the end), IOI is read as JOI, and the spaces are put back:

```
COPIII SE ÎNTÂLNESC JOI LA ORA ȘAPTE ÎN FAȚA ȘCOLII
```

This is the original message, with all its diacritics.

3.4 Other checks of the input

Listing 3.5 shows the remaining checks: a wrong number in the menu, and a cryptogram with the pair CC, which no Playfair encryption can produce. After that the cryptogram is typed in lower case, and the program accepts it, because every letter is converted to a capital.

Listing 3.5 – Checks of the menu and of the cryptogram

```
Playfair cipher for Romanian (31 letters, J is written as I)
Operation (1 - encrypt, 2 - decrypt, 0 - exit): 5
Type 1, 2 or 0.
Operation (1 - encrypt, 2 - decrypt, 0 - exit): 2
Key: Securitatea informației
Matrix:
    S E C U R I
    T A N F O M
    Ț Ă Â B D G
    H Î K L P Q
    Ș V W X Y Z
Cryptogram: ABCC DE
A cryptogram cannot have a pair of equal letters: CC
Cryptogram: rnqruzseavfabkaceurmuqnmeovthoavfotătvrnquuz
Decrypted pairs: CO PI IX IS EÎ NT ÂL NE SC IO IL AO RA ȘA PT EÎ NF AȚ AȘ CO LI IX
Decrypted message: COPIIXISEÎNTÂLNESCIOILAORAȘAPTEÎNFAȚAȘCOLIIX
Remove the filler letters X/Q and add the spaces by hand.

Operation (1 - encrypt, 2 - decrypt, 0 - exit): 0
```

CONCLUSIONS

The Playfair cipher was implemented in Python for the Romanian alphabet. The main difficulty was that 31 letters do not fit into a 5×5 matrix. A 5×6 matrix with J written as I holds the other 30 letters with no free cells, so a cryptogram never contains anything but letters, and the J is easy to restore after decryption. The rules of the cipher did not have to change; only the shifts are taken modulo 6 in a row and modulo 5 in a column.

The implementation was checked on the example of the guide, where it reproduced all 230 letters of the cryptogram, and on 5000 random Romanian texts, where decryption always returned the prepared text. The Romanian example covered the cases that the program has to handle: three equal letters in a row, a text that becomes odd after a filler is inserted, the letter J and the letters with diacritics.

The example also showed the strength and the weakness of the cipher. The same letter I was encrypted in three different ways, so the frequencies of single letters no longer reveal the plaintext as they did in Lab 2. But every repeated pair of the message (CO, EÎ, IX) gave the same pair in the cryptogram, so Playfair remains a monoalphabetic cipher on pairs of letters, and a long enough cryptogram can be broken by the frequency analysis of pairs.

REFERENCES

1. ZGUREANU, A. Lucrarea de laborator nr. 3. Cifruri polialfabetice. Faculty of Computers, Informatics and Microelectronics, Technical University of Moldova, 2022.

2. KAHN, D. The Codebreakers: The Story of Secret Writing. New York: Macmillan, 1967.

3. Python Software Foundation. unicodedata – Unicode Database [online]. Available: https://docs.python.org/3/library/unicodedata.html